気象庁データで 10 年間の猛暑日を可視化する：Python スクレイピング入門

komachan

2026-05-06

気象庁データで 10 年間の猛暑日を可視化する：Python スクレイピング入門

はじめに

近年の夏は年々厳しさを増している。本当に猛暑日（最高気温 35°C 以上）は増えているのか——そんな疑問を実際のデータで確かめようと、気象庁（JMA）の公開統計データを使って 2016〜2025 年の 10 年間分を全国の観測地点について集計する Python プロジェクトを作成した。

この記事では、プロジェクトの設計思想から実装上の落とし穴と解決策まで、手を動かしながら学べる形でまとめる。

プロジェクト概要

項目	内容
対象期間	2016〜2025 年（10 年間）
観測地点数	気象官署 約 160 点 ＋ AMeDAS 約 1,519 点
猛暑日の定義	日最高気温 35°C 以上（気象庁公式定義）
データソース	`data.jma.go.jp/obd/stats/etrn/`
成果物	統計 CSV、折れ線グラフ、全国インタラクティブマップ

主な最終結果（抜粋）：

- **全国平均の猛暑日数**：2016 年の 3.2 日/観測点から 2025 年には 11.3 日/観測点へと、10 年で**約 3.5 倍**に増加
- **最多猛暑日の観測点**：日田（大分県）の年平均 35.3 日
- **都道府県別 1 位**：埼玉県（22.9 日/年）

システム設計

```
hot_days/
```

```
├── pyproject.toml            # uv によるプロジェクト定義
├── src/jma_hot_days/
│   ├── fetch.py              # HTTP クライアント・キャッシュ
│   ├── stations.py           # 観測地点カタログ構築
│   ├── annual.py             # 年別猛暑日ページのパース
│   ├── aggregate.py          # 集計・サマリー
│   ├── coords.py             # 緯度経度の紐付け
│   ├── report.py             # グラフ・地図・レポート生成
│   └── cli.py                # CLIエントリーポイント
└── data/
    ├── raw/                  # ダウンロードキャッシュ
    ├── interim/              # パース結果
    └── processed/            # 最終集計
```

CLI は 4 つのサブコマンドを持つ：

```
uv run jma-hot-days catalog  # 観測地点一覧を取得
uv run jma-hot-days fetch    # 年別データをダウンロード・パース
uv run jma-hot-days report   # 集計とグラフ・地図を生成
uv run jma-hot-days all      # 上記 3 つを一括実行
```

Step 1：プロジェクトのセットアップ

uv を使って仮想環境ごとプロジェクトを管理する。

```
uv init hot_days
cd hot_days
uv add httpx beautifulsoup4 lxml pandas pyarrow \
       tenacity tqdm matplotlib folium tabulate
```

pyproject.toml のポイントは hatchling ビルドと src レイアウト：

```
[project]
name = "jma-hot-days"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "httpx>=0.28",
    "beautifulsoup4>=4.13",
    ...
]

[project.scripts]
jma-hot-days = "jma_hot_days.cli:main"

[tool.hatch.build.targets.wheel]
packages = ["src/jma_hot_days"]
```

Step 2：観測地点カタログの構築（`stations.py`）

気象庁のイメージマップ型ページから全 47 都道府県の観測地点を取得する。

```
https://www.data.jma.go.jp/obd/stats/etrn/select/prefecture00.php
  → 各都道府県ページ（47件）
    → 各観測地点リンク（block_no を取得）
```

エンコーディングの罠

気象庁の HTML はページによって UTF-8 と Shift_JIS が混在する。最初に Shift_JIS で試すと、UTF-8 ページが文字化けする。正しい順序：

```python
def _decode(raw: bytes) -> str:
    try:
        return raw.decode("utf-8")
    except UnicodeDecodeError:
        return raw.decode("shift_jis", errors="replace")
```

気象官署 vs AMeDAS

`block_no` の桁数で種別を判定できる。

種別	block_no	地点数
気象官署（有人）	5 桁（47xxx）	約 160
AMeDAS（自動）	4 桁以下	約 1,519

Step 3：年別猛暑日データの取得（`annual.py`）

正しい URL パラメータを探す

気象庁の観測データページは `view` パラメータで表示内容が変わる。デフォルト（`view=a1`）は平均値・極値で、猛暑日数は **`view=a2`** の「階級別日数」ページに掲載されている。

```
# 気象官署の場合
https://www.data.jma.go.jp/obd/stats/etrn/view/annually_s.php
  ?prec_no=&block_no=47784&view=a2

# AMeDASの場合
https://www.data.jma.go.jp/obd/stats/etrn/view/annually_a.php
  ?prec_no=&block_no=1058&view=a2
```

複雑な HTML テーブルのパース

view=a2 のテーブルヘッダーは rowspan/colspan を多用する 2 段構造になっている。Pandas の read_html では正しく処理できないため、自前でグリッドを展開する。

```
def _expand_header_grid(header_rows) -> list[list[str]]:
    """rowspan/colspan を展開して 2 次元グリッドに変換する。"""
    grid = []
    occupied: dict[tuple[int,int], str] = {}
    for ri, row in enumerate(header_rows):
        if ri >= len(grid):
            grid.append([])
        ci = 0
        for cell in row.find_all(["th", "td"]):
            while (ri, ci) in occupied:
                grid[ri].append(occupied.pop((ri, ci)))
                ci += 1
            text = cell.get_text(strip=True)
            rs = int(cell.get("rowspan", 1))
            cs = int(cell.get("colspan", 1))
            for r in range(ri, ri + rs):
                for c in range(ci, ci + cs):
                    if r == ri and c == ci:
                        continue
                    occupied[(r, c)] = text
                    if r >= len(grid):
                        grid.append([])
            grid[ri].append(text)
            ci += cs
    return grid
```

全角・半角パレンの両対応

気象官署ページは「各階級の日数（最高）」（全角丸括弧）、AMeDAS ページは「各階級の日数(最高)」（半角丸括弧）と表記が異なる。正規表現で両方にマッチさせる：

```
HOT_HIGH_GROUP_RE = re.compile(r"各階級の日数[（(]最高[）)]")
HOT_THRESHOLD_RE  = re.compile(r"35")
```

この 1 行の修正で AMeDAS データも正しく取得できるようになった。

丁寧なレート制限

気象庁サーバへの負荷軽減のため、リクエスト間隔を 1 秒以上確保し、ディスクキャッシュと tenacity によるリトライを組み合わせた：

```
def fetch_cached(url: str, cache_path: Path, client: httpx.Client) -> bytes:
    if cache_path.exists():
        return cache_path.read_bytes()
```

```
    time.sleep(1.0)  # 1 秒の待機
    resp = client.get(url)
    resp.raise_for_status()
    cache_path.parent.mkdir(parents=True, exist_ok=True)
    cache_path.write_bytes(resp.content)
    return resp.content
```

Step 4：座標の紐付け（`coords.py`）

地図描画には各観測地点の緯度経度が必要だが、気象庁のスクレイピング結果には座標が含まれない。

JMA は AMeDAS 地点テーブルを JSON で公開しており、ここから座標を取得できる：

`https://www.jma.go.jp/bosai/amedas/const/amedastable.json`

同名地点問題（重大なバグ）

全国には同じ漢字を持つ観測地点が複数存在する。例：

地点名	所在
山口	山口県（正）
山口	北海道石狩地方（AMeDAS）
高松	香川県（正）
高松	北海道渡島地方（AMeDAS）

名前だけでマッチングすると、AMeDAS の JSON は北海道（コード 1xxxx）から始まるため、**本州・四国の地点が北海道の座標でプロットされてしまう**。

解決策：`prec_no` + 地点名でのマッチング

AMeDAS コードの先頭 2 桁は気象庁の地方コード（`prec_no`）と一致する。これを使って厳密マッチングを行い、一致しない場合は地域間のコード差異を吸収するフォールバックマップを用意した：

```
# 気象庁データと AMeDAS コードで prec_no が異なるケース
_PREC_FALLBACK_MAP = {
    "91": ("91", "93", "94"),  # 沖縄：本島 / 大東島 / 先島諸島
    "23": ("23", "24"),        # 渡島・檜山地方
    "11": ("11", "13"),        # 宗谷地方
}
```

この修正により、山口（山口県）・高松（香川県）・大津（滋賀県）・金山（岐阜県）・新城（愛知県）など 22 地点が正しい座標にプロットされるようになった。

Step 5：集計と可視化（`aggregate.py`, `report.py`）

集計

```
def per_station_summary(df: pd.DataFrame, n_years: int) -> pd.DataFrame:
    grp = df.groupby(["prec_no", "block_no", "name", "kind", "pref_name"])
    agg = grp["hot_days"].agg(
        total_hot_days="sum",
        max_hot_days="max",
        years_with_data="count",
    ).reset_index()
    agg["mean_hot_days"] = agg["total_hot_days"] / n_years
    agg["coverage"] = agg["years_with_data"] / n_years
    return agg.sort_values("mean_hot_days", ascending=False)
```

日本語フォントの対応

macOS の場合、matplotlib は標準でヒラギノ角ゴシックを認識しないため、手動でフォントを指定する。`japanize-matplotlib` は Python 3.13 非対応のため使用できない。

```
def _configure_cjk_font() -> None:
    candidates = [
        "Hiragino Sans",
        "Hiragino Kaku Gothic ProN",
        "Yu Gothic",
        "Noto Sans CJK JP",
    ]
    available = {f.name for f in matplotlib.font_manager.fontManager.ttflist}
    for name in candidates:
        if name in available:
            matplotlib.rcParams["font.family"] = name
            return
```

生成される成果物

1. **`national_trend.png`** —全国年別猛暑日数の推移（折れ線グラフ）
2. **`top_stations.png`** —猛暑日数上位 20 観測点の棒グラフ
3. **`prefecture_summary.png`** —都道府県別ヒートマップ
4. **`hot_days_map.html`** —Folium によるインタラクティブ全国マップ

主な結果

全国年別猛暑日数の推移

年	観測点あたり平均猛暑日数
2016	3.2 日
2017	2.1 日
2018	**7.0 日**（40.7°C を記録した夏）
2019	3.5 日
2020	5.1 日
2021	2.7 日
2022	4.1 日
2023	**7.7 日**
2024	**11.2 日**
2025	**11.3 日**

2023 年以降の急増が顕著で、2025 年は 2016 年比で約 **3.5 倍**に達する。

猛暑日数の多い観測地点 TOP 10

順位	地点	都道府県	年平均	最大値
1	日田	大分県	35.3 日	62 日
2	多治見	岐阜県	34.0 日	59 日
3	京都	京都府	32.8 日	61 日
4	久留米	福岡県	30.9 日	54 日
5	鳩山	埼玉県	30.4 日	51 日
6	豊田	愛知県	30.2 日	57 日
7	太宰府	福岡県	30.1 日	62 日
8	館林	群馬県	30.0 日	54 日
9	枚方	大阪府	29.4 日	51 日
10	桐生	群馬県	29.2 日	58 日

都道府県別 TOP 5

順位	都道府県	年平均（観測点あたり）
1	埼玉県	22.9 日
2	京都府	19.6 日
3	大阪府	15.9 日
4	香川県	15.6 日
5	福岡県	15.5 日

ハマったポイントまとめ

問題	原因	解決策
データが 0 件	デフォルト URL は平均値ページ	`&view=a2` を付加
文字化け	UTF-8/Shift_JIS 混在	UTF-8 を先に試みる
AMeDAS データが 0 件	全角/半角丸括弧の不一致	正規表現で両方にマッチ
60 件の偽観測点	“全地点”のリンク	`len(block_no) <= 2` でフィルタ
日本語フォント不表示	japanize-matplotlib 非対応（Py3.13）	ヒラギノを rcParams で直接指定
座標の誤プロット	同名地点の名前一致で北海道が優先	`prec_no` + 名前の複合キーでマッチング

再現方法

```
# 1. リポジトリを取得し、依存関係をインストール
git clone <repo>
cd hot_days
uv sync

# 2. 全パイプラインを実行（初回は数時間かかる）
uv run jma-hot-days all

# 3. 成果物を確認
open reports/hot_days_map.html
open reports/hot_days_report.md
```

おわりに

気象庁のオープンデータを使うことで、「最近の夏は本当に暑くなっているのか」という問いに対して具体的な数字で答えることができた。2016〜2025 年の 10 年間で、全国平均の猛暑日数は約 3.5 倍に増加しており、特に 2023 年以降の急増が際立っている。

Python によるウェブスクレイピング・データ処理・可視化のパイプラインとして、同じアプローチを他の気象指標（熱帯夜数、降水量など）にも応用できる。コード全体は約 1,000 行程度にまとまっており、モジュールごとに役割が分離されているため、部分的な改造も容易だ。

興味がある方はぜひ手元で試してみてほしい。

猛暑日ゼロの地点：日本にはまだ「涼しい場所」がある

猛暑日が増え続ける一方で、2016〜2025 年の 10 年間を通じて**一度も猛暑日を記録しなかった観測地点が 153 か所**存在する。全国 1,321 地点の 11.6% に相当する。

なぜ猛暑日ゼロなのか

大きく 4 つのタイプに分けられる。

1. 北海道（73 件）—オホーツク海・太平洋の冷気

地域	代表地点
宗谷地方	稚内、宗谷岬、礼文
根室・釧路地方	根室、釧路
日高・胆振地方	えりも岬、室蘭、苫小牧
後志・檜山地方	倶知安、寿都、江差

北海道の太平洋・オホーツク海沿岸は、夏に海霧（やませ）が発生しやすく、真夏でも最高気温が 30°C に達しないこともある。胆振地方・釧路地方の全観測地点がゼロという結果は、この気候帯の涼しさを裏付けている。

2. 高原・山岳（24 件）—標高が最大の防護壁

地点	標高	所在
富士山	3,776m	山梨県・静岡県
奥日光（日光）	約 1,272m	栃木県
野辺山	約 1,350m	長野県
菅平	約 1,340m	長野県
草津	約 1,220m	群馬県
軽井沢	約 950m	長野県
那須高原	約 740m	栃木県
阿蘇山	約 1,142m	熊本県

標高が高くなるほど気温は低下し（100m で約 0.6°C の低下）、軽井沢クラスの 950m でも平地より 5〜6°C 低い。長野県はこのカテゴリに 11 地点が集中しており、高原避暑地の涼しさがデータでも確認できる。

3. 沿海・離島（29 件）—海洋性気候と海風　意外なのが**沖縄の離島**（宮古島、南大東島など）と**東京の島嶼**（父島、三宅島、南鳥島）だ。南国のイメージとは裏腹に、猛暑日がゼロである。

海洋に囲まれた島は気温の日較差が小さく、最高気温も 33〜34°C 前後で頭打ちになることが多い。周囲の海水温が大気を冷やし、35°C 超えを抑制する。

千葉・勝浦も有名な涼しい地点だ。太平洋に面しており、夏でも南東からの涼しい海風が吹き込む。「勝浦の涼しさ」は地元では常識だが、10 年データでも猛暑日ゼロが裏付けられた。

室戸岬（高知）・石廊崎（静岡）など突端の岬も同様のメカニズムで冷やされる。

4. 東北・山間部（24 件）—北方と標高の複合効果　青森・岩手・秋田・福島の山間部がランクインしている。

地点	所在	特徴
酸ケ湯	青森県	豪雪地帯・標高 890m
薮川	岩手県	内陸盆地・標高約 780m
桧枝岐	福島県	奥会津・標高約 770m
肘折	山形県	豪雪地帯・標高約 300m

これらはいずれも標高があるか、または冷気の溜まりやすい地形に位置する。

猛暑日ゼロ有人観測点 一覧（気象官署 28 地点）

有人観測点（気象官署）でゼロは 28 地点。避暑地・島嶼・北海道の沿岸都市が並ぶ。

地点	所在	カテゴリ
昭和	南極	南極
稚内	宗谷地方（北海道）	北海道
北見枝幸	宗谷地方（北海道）	北海道
羽幌	留萌地方（北海道）	北海道
倶知安	後志地方（北海道）	北海道
寿都	後志地方（北海道）	北海道
浦河	日高地方（北海道）	北海道
室蘭	胆振地方（北海道）	北海道
苫小牧	胆振地方（北海道）	北海道
釧路	釧路地方（北海道）	北海道
根室	根室地方（北海道）	北海道
江差	檜山地方（北海道）	北海道
奥日光（日光）	栃木県	高原・山岳
勝浦	千葉県	沿海
三宅島	東京都	離島
南鳥島	東京都	離島
父島	東京都	離島
石廊崎	静岡県	沿海
富士山	静岡県・山梨県	高原・山岳
軽井沢	長野県	高原・山岳
室戸岬	高知県	沿海
宮古島	沖縄県	離島
南大東（南大東島）	沖縄県	離島
平戸	長崎県	沿海
雲仙岳	長崎県	高原・山岳
沖永良部	鹿児島県	離島
阿蘇山	熊本県	高原・山岳
軽井沢	長野県	高原・山岳

まとめ

猛暑日ゼロの地点には、「海洋性気候」「高標高」「北方冷涼帯」という 3 つの共通原理がある。日本列島は南北に長く、同じ夏でも北海道の釧路・稚内と、埼玉の熊谷・鳩山の間には実に 35 日以上の猛暑日数差がある。近年の急速な温暖化の中でも、これらの「涼しい場所」がどこまで猛暑日ゼロを維持できるかは、今後の重要な観測指標になるだろう。